



ZenithWeb — Manual de Instalación

Sistema de Gestión de Stock de Paneles Solares

Stack: **Node.js + Express** (backend) · **React + Vite** (frontend) · **PostgreSQL** (base de datos)

Este manual está pensado para instalar y ejecutar el proyecto **desde cero en una PC nueva** (por ejemplo, la de la profesora). Seguir los pasos en orden.

1. Requisitos previos

Antes de empezar, instalar estos programas (todos gratuitos):

Programa	Versión recomendada	Para qué sirve	Descarga
Node.js	18 LTS o superior	Ejecutar backend y frontend	https://nodejs.org
PostgreSQL	14 o superior	Base de datos	https://www.postgresql.org/download
Git *(opcional)*	última	Clonar el repositorio	https://git-scm.com

Al instalar PostgreSQL, anotá la **contraseña del usuario postgres** que te pide el instalador: la vas a necesitar más adelante.

Verificar que quedó todo instalado (abrir una terminal / PowerShell y ejecutar):

```
node -v      # debe mostrar v18.x o superior
npm -v       # debe mostrar un número de versión
psql --version # debe mostrar la versión de PostgreSQL
```

2. Obtener el proyecto

Opción A — con Git:

```
git clone https://github.com/GastonResoagli/zenithweb.git
cd zenithweb
```

Opción B — sin Git: descargar el **.zip** del proyecto, descomprimirlo y abrir una terminal dentro de la carpeta **zenithweb**.

Estructura del proyecto

```
zenithweb/
├─ index.js           ← punto de entrada del backend
├─ package.json       ← dependencias del backend
├─ backup.sql         ← base de datos (datos + estructura)
├─ src/              ← código del backend (controllers, services, etc.)
├─ client/            ← aplicación frontend (React)
│   └─ package.json   ← dependencias del frontend
└─ MANUAL_INSTALACION.md ← este archivo
```

3. Crear y cargar la base de datos

3.1. Crear la base de datos vacía

Abrir una terminal y entrar a PostgreSQL (te va a pedir la contraseña de `postgres`):

```
psql -U postgres
```

Dentro de `psql`, crear la base llamada `zenithweb` y salir:

```
CREATE DATABASE zenithweb;
\q
```

3.2. Restaurar los datos desde `backup.sql`

Desde la carpeta del proyecto (donde está `backup.sql`), ejecutar:

```
psql -U postgres -d zenithweb -f backup.sql
```

Esto crea todas las tablas (productos, usuarios, ventas, etc.) y carga datos de prueba, **incluidos los usuarios para iniciar sesión** (ver sección 6).

4. Configurar y ejecutar el BACKEND

4.1. Crear el archivo `.env`

En la **raíz del proyecto** (misma carpeta que `index.js`), crear un archivo llamado `.env` con este contenido. Reemplazá `TU_PASSWORD` por la contraseña de PostgreSQL que definiste en la instalación:

```
DB_USER=postgres
DB_HOST=localhost
DB_NAME=zenithweb
DB_PASSWORD=TU_PASSWORD
DB_PORT=5432
PORT=3000
JWT_SECRET=zenith_secret_key
```

⚠ **Importante:** el backend debe correr en el **puerto 3000**, porque el frontend está configurado para conectarse a `http://localhost:3000`. No cambiar `PORT`.

4.2. Instalar dependencias y arrancar

Desde la raíz del proyecto:

```
npm install    # instala las dependencias del backend (solo la primera vez)
npm start      # levanta el servidor
```

Si todo salió bien, vas a ver en la terminal:

```
servidor corriendo en puerto 3000
```

💡 Dejá esta terminal abierta. Para desarrollo con recarga automática podés usar `npm run dev` en lugar de `npm start`.

5. Configurar y ejecutar el FRONTEND

Abrir una segunda terminal (sin cerrar la del backend) y ejecutar:

```
cd client
npm install    # instala las dependencias del frontend (solo la primera vez)
npm run dev    # levanta la aplicación web
```

La terminal va a mostrar una dirección local, normalmente:

```
→ Local:    http://localhost:5173/
```

Abrir esa dirección en el navegador (Chrome / Edge / Firefox).

6. Iniciar sesión (usuarios de prueba)

La pantalla de login pide **correo** y **contraseña**. Estos usuarios ya vienen cargados en la base de datos:

Rol	Correo	Contraseña	Qué puede hacer
Gerente	gerente@zenith.com	gerente123	Acceso total (incluye gestión de roles)
Operador de Stock	operador@zenith.com	operador123	ABM de productos + registrar ventas
Vendedor	vendedor@zenith.com	vendedor123	Solo lectura del catálogo de productos

Para ver **todas las funcionalidades**, iniciar sesión como **Gerente**.

7. Resumen rápido (chuleta)

```
# 1) Base de datos (una sola vez)
psql -U postgres -c "CREATE DATABASE zenithweb;"
psql -U postgres -d zenithweb -f backup.sql

# 2) Backend → terminal 1, en la raíz del proyecto
npm install
npm start

# 3) Frontend → terminal 2
cd client
npm install
npm run dev

# 4) Abrir el navegador en http://localhost:5173
#   Login: gerente@zenith.com / gerente123
```

8. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
error: password authentication failed for user "postgres"	Contraseña incorrecta en <code>.env</code>	Corregir <code>DB_PASSWORD</code> en el archivo <code>.env</code>
database "zenithweb" does not exist	No se creó la base	Ejecutar el paso 3.1 (<code>CREATE DATABASE zenithweb;</code>)

Síntoma	Causa probable	Solución
El backend no inicia / <code>ECONNREFUSED</code>	PostgreSQL no está corriendo	Iniciar el servicio de PostgreSQL en Windows (Servicios → <code>postgresql-x64</code>)
La web carga pero el login falla con "Error de red"	El backend no está levantado o no está en el puerto 3000	Verificar la terminal 1: debe decir <code>"servidor corriendo en puerto 3000"</code> *
<code>'psql' no se reconoce como comando</code>	PostgreSQL no está en el PATH	Usar la terminal SQL Shell (psql) que instala PostgreSQL, o agregar su carpeta <code>bin</code> al PATH
<code>Port 5173 is in use</code>	El frontend ya está corriendo en otra terminal	Vite ofrece otro puerto automáticamente; usar el que indique
Credenciales incorrectas al loguear	La base no se restauró bien	Repetir el paso 3.2

9. Notas técnicas

- **Backend:** puerto `3000` (configurable en `.env` , pero el frontend espera el 3000).
- **Frontend:** puerto `5173` (Vite, por defecto).
- **Base de datos:** PostgreSQL en `localhost:5432` .
- La **autenticación** usa JWT; el token se guarda en el navegador y define el rol del usuario.
- Los **productos eliminados** no se borran físicamente (baja lógica con `estado = false`) para preservar el historial de ventas.

Integrantes: Matías Lago · Gastón Resoagli

Uso académico.